A sharper exponent for deterministic integer 3SUM

A refinement of the existing ternary construction

20 October 2026

Abstract

We refine the numerical certificates and exponent margins in the published ternary 3SUM formalization, establishing the exact rational exponent $4800699/2500000 = 1.9202796$, below the listed bound 1.9203. The probability profile and algorithmic construction are inherited; the contribution is a more precise proof of their running time. Exact rational logarithm enclosures put every principal cost below 1.92027946. The finite-size and execution overheads fit within successive strict margins ending at 1.9202796. We provide the source patch, certificates, and reproduction instructions. The construction and its original formalization remain credited to their authors.

Verification status. The complete 1,198-module Lean source build and full-source axiom audit passed in GitHub Actions with Lean v4.33.1. The audit checked 43,758 submitted declarations. The exported theorem depends only on the permitted axioms `propext`, `Classical.choice`, and `Quot.sound`. The independent exact rational checks also passed on the same source snapshot. Official leaderboard acceptance and independent Nanoda verification are not established.

1 Problem and scope

The benchmark asks whether an input list of n integers contains three distinct positions whose values sum to zero. Equal values at different positions are permitted. For every fixed natural number κ, the input magnitudes are at most n^κ. The conclusion supplies a single finite deterministic word-RAM program for that κ, independent of n, the input values, and the admissible word width. The minimum word width is $b(\lfloor \log_2 n \rfloor + 1)$ for a fixed constant b.

The underlying statement, arithmetic instructions, indirect memory access, branch semantics, and compiler are the published benchmark definitions. This refinement concerns their asymptotic upper bound. It does not establish a practical implementation, crossover size, or measured speedup.

The base source is the immutable revision `4a4a98e6eca4fc16817f0d2074026fdaece52efd` of `fringewidth/three-sum-lean`, the source linked by the 1.9203 leaderboard entry. The refined bound is

$$\boxed{\text{3SUM} \in O\Big(n^{4800699/2500000}\Big) = O(n^{1.9202796}).}$$

The exponent difference is exactly 0.0000204. This is a tightening of the analysis of the existing construction; the finite constants chosen by its existence proofs may change when their admissibility conditions are tightened.

2 The inherited parameter profile

Use the exact decimal rationals

$$\pi = (0.81204903,\ 0.15860350,\ 0.02934747), \qquad \alpha = (0.90113327,\ 0.08908424,\ 0.00978249),$$

$$\delta = 0.01344845, \qquad y = 0.18162395, \qquad z = 0.07523953.$$

They satisfy $\sum \pi_i = \sum \alpha_i = 1$ and

$$\alpha_0 = \pi_0 + \pi_1/2 + \pi_2/3, \quad \alpha_1 = \pi_1/2 + \pi_2/3, \quad \alpha_2 = \pi_2/3.$$

Writing $\mu = \pi_1 + 2\pi_2$, define the five quantities already used in the source, with all logarithms in base two:

$$H = -\sum_{i=0}^{2} \pi_i \log_2 \pi_i, \qquad A = -\sum_{i=0}^{2} \alpha_i \log_2 \alpha_i,$$
$$G = \pi_1 + \pi_2 \log_2 3,$$
$$B = \log_2(1 + y + y^2) - (\mu - \delta)\log_2 y,$$
$$Q = \pi_1 \log_2(1 + z) + \pi_2 \log_2(1 + z + z^2) - \delta \log_2 z.$$

The principal cost ledger in `TernaryParameters.lean` is

$$\max\left\{\frac{2B}{H},\ 2 - \frac{G}{H} + \frac{2Q}{H},\ \frac{2A}{H},\ 1 + \frac{G}{H},\ 2 - \frac{G}{H},\ 1\right\}. \tag{1}$$

The first two entries are the active costs, for the stored bank and queries. The other entries retain substantial room. The numerical ledger alone is not a runtime theorem: the later executable cost proofs are also required.

3 Exact logarithm and entropy certificates

In this section ln denotes the natural logarithm. For positive rational x, put $t = (x-1)/(x+1)$. The finite expansion

$$S_N(t) = 2\sum_{j=0}^{N-1} \frac{t^{2j+1}}{2j+1}$$

has the rigorous enclosure used by the existing Mathlib theorem

$$|\ln x - S_N(t)| \le \frac{2|t|^{2N+1}}{1-t^2}.$$

This deliberately uses the precise remainder provided by `Real.sum_range_sub_log_div_le`. All quantities on the right are rational. Arguments are normalized by powers of two, and the resulting logarithm bounds are combined with the enclosure of $\ln 2$ to obtain the base-two entropy bounds.

We increase N from 8 to 16 and round logarithm enclosures outward on a 10^{-12} grid. The entropy intervals use a 10^{-11} grid. Rational arithmetic, rather than floating-point evaluation, certifies the following inclusive intervals:

Quantity	Lower endpoint	Upper endpoint
H	0.81463563795	0.81463563796
A	0.51142981001	0.51142981003
G	0.20511813944	0.20511813945
B	0.78216403986	0.78216403988
Q	0.07008744368	0.07008744369

For the direct logarithm estimates, the largest coarse remainder is below 4.048×10^{-16}. The independent check also verifies that each declared rational interval contains the exact enclosure used by the Lean proof, not only a high-precision numerical approximation to the logarithm.

Since $H > 0$, cross-multiplication proves all ledger entries strictly below $c = 1.92027946$. For example, the two active inequalities reduce to

$$cH - 2B > 0, \qquad G - 2Q - (2 - c)H > 0.$$

Each is a linear rational inequality after inserting the appropriate interval endpoints. The following upper bounds are rounded upward for display:

Cost	Certified upper bound
$2B/H$	1.92027945610
$2 - G/H + 2Q/H$	1.92027938746
$2A/H$	1.25560382141
$1 + G/H$	1.25179126704
$2 - G/H$	1.74820873299
Constant term	1

In particular, the exact bank bound is 156432807976/81463563795, which is strictly below 1.92027946. The unrounded exact fractions and positive cross-multiplication residuals are included in the machine-readable certificate.

4 Preserving the runtime overheads

The refinement uses the following successive exponents:

Stage	Exact decimal rational	Rational form
Principal certificate	1.92027946	96013973/50000000
Finite-block certificate	1.92027957	192027957/100000000
Frontend budget	1.92027958	96013979/50000000
Executed colored solver	1.92027959	192027959/100000000
Final benchmark	1.92027960	4800699/2500000

The source already chooses a fixed $K \geq 10^8$. Thus

$$\frac{10}{K} \leq 10^{-7} < 1.1 \times 10^{-7} = 1.92027957 - 1.92027946.$$

This covers both the $8/K$ and $10/K$ additions in the finite-block rate proofs. Their eventual convergence arguments then select a sufficiently large fixed block size.

The source's separate allowance is

$$\varepsilon_K = \frac{1}{100K^2} \leq 10^{-18} < 10^{-8}.$$

Every subsequent gap in the table is 10^{-8}. Hence the frontend and executed-program inequalities retain strict room. The auxiliary runtime parameter lemma also remains valid: its left side is a full exponent strictly below the frontend budget, plus $\varepsilon_K + 10^{-8}$, and the final target is exactly 2×10^{-8} above the frontend budget.

Finally, the existing `PowerBound` predicate permits arbitrarily small positive slack. The already proved hybrid and compiler bridges convert the actual executed-program bound at 1.92027959 into the exact rational benchmark at 1.9202796. This step does not assert that a $n^{a+o(1)}$ estimate implies an exact $O(n^a)$ estimate at its endpoint.

5 Source changes and reproducibility

The patch changes eleven files. It tightens logarithm and entropy intervals, the stage exponents, and the explicitly stated differences between them; it then updates the final theorem and its export. The immutable benchmark statement, machine and compiler sources, dependency manifest, and probability profile are preserved. Historical milestone constants that are unrelated to the exported theorem are retained.

The final export is

```
theorem Race.bound :
    EndStatement.ThreeSum.SolvedInTime
      ((4800699 : Rat) / 2500000) := by
  exact ThreeSum.solvedInTime_4800699_2500000
```

Use Lean `v4.33.1` and Mathlib revision `0df444a360eaa60ab8c11dca51a86af692955474`. The package's instructions apply the patch to the pinned base revision, check exact rational certificates, build the complete submitted source, and perform the permitted-axiom audit. A successful local check and an official leaderboard acceptance are separate events.

6 What the refinement leaves open

For the fixed profile above, the same entropy certificate gives

$$\frac{156432807972}{81463563796} \le \frac{2B}{H} \le \frac{156432807976}{81463563795}.$$

Thus this particular bank-cost expression lies between 1.92027945602 and 1.92027945610. The remaining gap to the exported exponent is principally the retained runtime margins. More decimal precision in the same logarithm estimates cannot remove those margins or reduce this fixed cost expression substantially. Further progress may come from adjusting the parameter profile, proving smaller overheads, or changing the construction. These observations do not prove global optimality of the profile or a lower bound for the 3SUM problem.

Attribution and sources

The algorithmic construction and full original formalization are due to `c0ldzy17` and the authors credited in the upstream `NOTICE` and `METHOD.md`. The previous margin refinement is published by Hrishik Sai Bojnal's `fringewidth` account. The inherited framework credits Josh Alman and Virginia Vassilevska Williams; classical subset convolution work by Björklund, Husfeldt, Kaski, and Koivisto; and Swapnil Jain's deterministic filtering work. The benchmark, compiler, and program logic originate in Anthropic's formalization, through the upstream source lineage. The present work claims the numerical refinement and its accompanying checks, not those prior constructions.

1. Big O Speedrun, 3SUM problem and leaderboard: `https://bigospeedrun.com/3sum`.
2. Official check of the base 1.9203 revision: `https://bigospeedrun.com/receipts/direct-issue-18-39196965603-1`.
3. Immutable proof source: `https://github.com/fringewidth/three-sum-lean/tree/4a4a98e6eca4fc16817f0d2074026fdaece52efd`.
4. Original construction repository: `https://github.com/c0ldzy17/three-sum-lean`.
5. J. Alman and V. Vassilevska Williams, *Truly Subquadratic 3SUM and Truly Subcubic APSP via Triangles in Sparse Lopsided Graphs*: `https://arxiv.org/abs/2610.06783`.
6. Refinement repository and verification records: `https://github.com/JoshuaQFEX/3sum-bound`.